

IMDB Movie Review Sentiment Classification

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.datasets import imdb

from tensorflow.keras.preprocessing.sequence import pad_sequences

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Embedding, GlobalAveragePooling1D, Dense

import matplotlib.pyplot as plt


print("TensorFlow version:", tf.__version__)

# Load the dataset

vocab_size = 10000

max_length = 250


print("Loading IMDB dataset...")

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=vocab_size)


# Pad sequences to ensure uniform input size

x_train = pad_sequences(x_train, maxlen=max_length)

x_test = pad_sequences(x_test, maxlen=max_length)


print(f"Training reviews: {len(x_train)}")

print(f"Test reviews: {len(x_test)}")

# Define the model architecture

embedding_dim = 16
```

```
model = Sequential([
    Embedding(vocab_size, embedding_dim, input_length=max_length),
    GlobalAveragePooling1D(),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])
```

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

```
model.summary()
```

```
# Train the model
```

```
epochs = 10
```

```
history = model.fit(
    x_train, y_train,
    epochs=epochs,
    batch_size=512,
    validation_split=0.2,
    verbose=1
```

```
)
```

```
# Visualize training results
```

```
acc = history.history['accuracy']
```

```
val_acc = history.history['val_accuracy']
```

```
loss = history.history['loss']
```

```
val_loss = history.history['val_loss']

epochs_range = range(1, epochs + 1)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

plt.plot(epochs_range, acc, label='Training Accuracy')

plt.plot(epochs_range, val_acc, label='Validation Accuracy')

plt.title('Accuracy')

plt.legend()

plt.subplot(1, 2, 2)

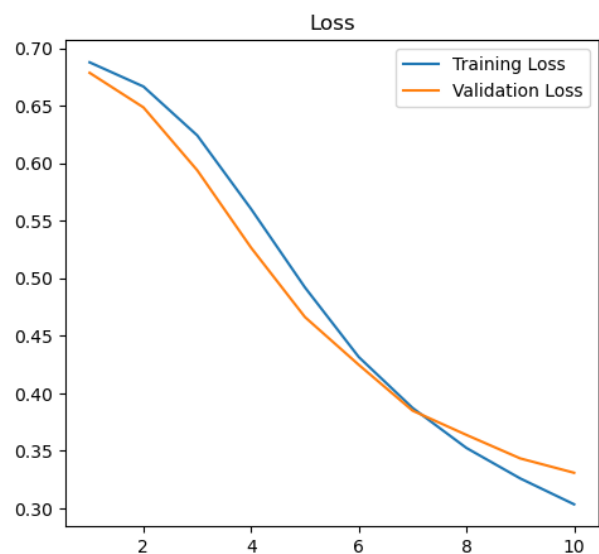
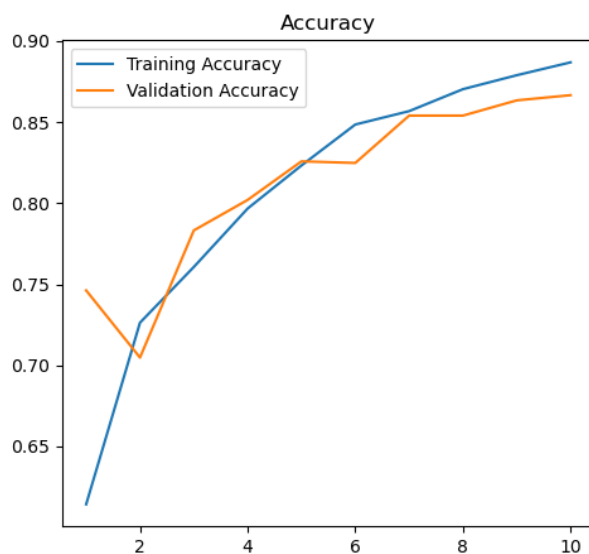
plt.plot(epochs_range, loss, label='Training Loss')

plt.plot(epochs_range, val_loss, label='Validation Loss')

plt.title('Loss')

plt.legend()

plt.show()
```



```
# Helper function to test your own reviews
```

```
word_index = imdb.get_word_index()
```

```
def predict_sentiment(review_text):
```

```
    # Preprocess the text to match model training
```

```
    words = review_text.lower().split()
```

```
    review_seq = [word_index.get(w, 0) + 3 for w in words]
```

```
    review_seq = [i if i < vocab_size else 0 for i in review_seq]
```

```
    padded = pad_sequences([review_seq], maxlen=max_length)
```

```
    prediction = model.predict(padded)[0][0]
```

```
    sentiment = "POSITIVE" if prediction > 0.5 else "NEGATIVE"
```

```
    return sentiment, prediction
```

```
# Example
```

```
my_review = "This movie was absolutely wonderful. The acting was great and the plot was gripping."
```

```
sentiment, score = predict_sentiment(my_review)
```

```
print(f"Review: {my_review}")
```

```
print(f"Sentiment: {sentiment} (Score: {score:.4f})")
```

Review: This movie was absolutely wonderful. The acting was great and the plot was gripping.

Sentiment: POSITIVE (Score: 0.6038)